Normalisation manuelle et vérification vidéo

Projet de suivi des joueurs de football
Amine - Séance du 19 octobre 2026 - Branche normalisation

Les étapes **A à D sont réalisées** dans le projet ouvert depuis le Bureau. Les traitements sont regroupés dans un module commun, la normalisation manuelle fonctionne dans la boucle vidéo et un programme de test permet de comparer les calculs à OpenCV.

Résultat principal : les 32 tests mathématiques réussissent. Sur les 450 images vidéo comparées, les valeurs de luminosité et les masques sont identiques à OpenCV. Le calcul manuel est cependant nettement plus lent dans les mesures indicatives réalisées. Aucun gain de détection des joueurs n'est démontré à ce stade.

Les quatre étapes réalisées

Étape	Travail effectué
A - Référence	Sauvegarde du code initial, compilation et lecture de 150 images de chacun des trois clips, sans fenêtres.
B - Organisation	Déplacement des deux fonctions manuelles dans un module partagé entre le programme vidéo et les tests.
C - Vérification	Tests de matrices simples, uniformes, aléatoires, vides et de sous-images ; comparaison aux fonctions OpenCV.
D - Vidéo	Normalisation manuelle par défaut sur chaque image ; modes OpenCV et aucune disponibles pour comparer.

Les fichiers ajoutés ou modifiés

Fichier dans le dépôt	Rôle
src/traitements_image.hpp	Nouveau : déclarations et contrat des fonctions.
src/traitements_image.cpp	Nouveau : calculs manuels commentés en français.
src/test_traitements.cpp	Nouveau : tests mathématiques et comparaison vidéo.
src/detourage_terrain.cpp	Appels du module, choix du mode, tests sans fenêtres et exports.
CMakeLists.txt	Bibliothèque commune, programme de test et intégration à CTest.
.vscode/launch.json	Sélection vidéo/mode et lancement des tests avec F5.
avancement/normalisation.md	Compte rendu des changements et résultats.

Dossier du projet : C:\Users\amine\Desktop\PROJET2A\S7-MultipistageFootball

Comprendre les changements de code

Un module commun pour les calculs

Les fonctions ne sont plus définies au début du programme de détourage. Le fichier .hpp annonce leur nom et leur paramètre ; le fichier .cpp contient le calcul. Le programme vidéo et le programme de test utilisent ainsi exactement la même implémentation.

```
void normaliserMinMax(cv::Mat& canal);
void egaliserHistogramme(cv::Mat& canal);
```

Le paramètre est un canal mono-canal 8 bits, de type **CV_8UC1**. Les pixels sont modifiés directement, sans changer les dimensions. Une matrice vide est acceptée ; un autre type est rejeté explicitement. Les parcours ligne par ligne fonctionnent aussi sur une sous-image non contiguë.

La normalisation min max reste calculée à la main

Une première passe cherche les valeurs minimale et maximale. Une seconde passe transforme chaque pixel selon la formule suivante, puis arrondit le résultat :

$$v \text{ nouveau} = \text{arrondi de } [255 \times (v - \text{minimum}) / (\text{maximum} - \text{minimum})]$$

Si maximum = minimum, le canal devient nul pour éviter une division par zéro. Cette convention correspond au résultat attendu pour la normalisation vers 0 à 255. Les commentaires du code expliquent les deux passes et la possibilité d'un écart d'arrondi d'un niveau avec OpenCV.

L'égalisation est conservée dans son rôle actuel

La fonction calcule manuellement l'histogramme de 256 niveaux, son cumul et une table de correspondance. Elle conserve une image uniforme. Son utilisation reste limitée à la luminosité du **rendu couleur redimensionné**. Elle ne modifie pas le masque de pelouse et n'est pas une détection de joueurs.

Le mode choisi concerne la boucle vidéo

Mode	Traitement de la luminosité à chaque image
manuel - défaut	Appel de normaliserMinMax().
opencv	Appel de cv::normalize ; référence historique de la boucle.
aucune	Le canal V de chaque image est conservé sans normalisation.

Pour comparer les variantes dans la même zone, le **contour initial du terrain conserve la normalisation manuelle historique** dans les trois modes. « Aucune » signifie donc aucune normalisation dans la boucle, et non dans l'ensemble du programme.

Les seuils HSV, le contour initial et l'égalisation du rendu sont conservés. Les nouvelles options permettent de limiter le nombre d'images et d'exporter des résultats sans ouvrir de fenêtres. Le CSV distingue le masque brut sur l'image entière de son rendu d'affichage.

Tests et résultats

Les tests mathématiques

32 vérifications sur 32 réussissent. Les cas couvrent les résultats connus à la main, les constantes 0/80/255, le vide, des données aléatoires reproductibles sur plusieurs plages, les lignes et colonnes, les sous-images non contiguës et le rejet des types invalides. La comparaison avec OpenCV accepte au maximum un niveau d'écart d'arrondi.

La compilation Release complète du projet réussit. CTest exécute le programme mathématique installé dans le projet. Les configurations Release et Debug sont vérifiées séparément.

La comparaison sur les vidéos

Les **150 premières images de chaque clip** sont lues séquentiellement à la résolution d'origine. Les deux versions reçoivent le même canal V, puis les masques utilisent les mêmes H, S et seuils. Les comparaisons portent sur l'image entière.

Vidéo	Dimensions	Images	Écart V max	Pixels masque différents
Clip 1	4096 × 1080	150	0	0
Clip 2	3840 × 1906	150	0	0
Clip 3	4096 × 1080	150	0	0

Total : 450 images comparées, aucun écart observé sur V ou sur les masques. Les tests sur plages restreintes complètent ces essais pour vérifier aussi une transformation min max non triviale.

Le temps de calcul est indicatif

Vidéo	Manuel en ms par appel	OpenCV en ms par appel
Clip 1	46,68	0,922
Clip 2	75,62	1,588
Clip 3	45,54	0,906

Le calcul manuel est nettement plus lent sur ces essais. Chaque mesure exclut décodage, copies, conversions, masques et captures. Une seule mesure est faite par fonction et par image, dans l'ordre manuel puis OpenCV, avec une charge du PC non isolée. Ces valeurs ne sont **ni des FPS globaux ni un benchmark contrôlé**.

La régression et les options du programme

Le programme installé a traité 150 images par clip dans les deux modes manuel et OpenCV : **900 traitements d'images**, soit plusieurs passages sur les mêmes images. Les **24 captures comparées** à la référence sont identiques pixel par pixel : images 0 et 149, trois clips, deux modes, rendus couleur et masque.

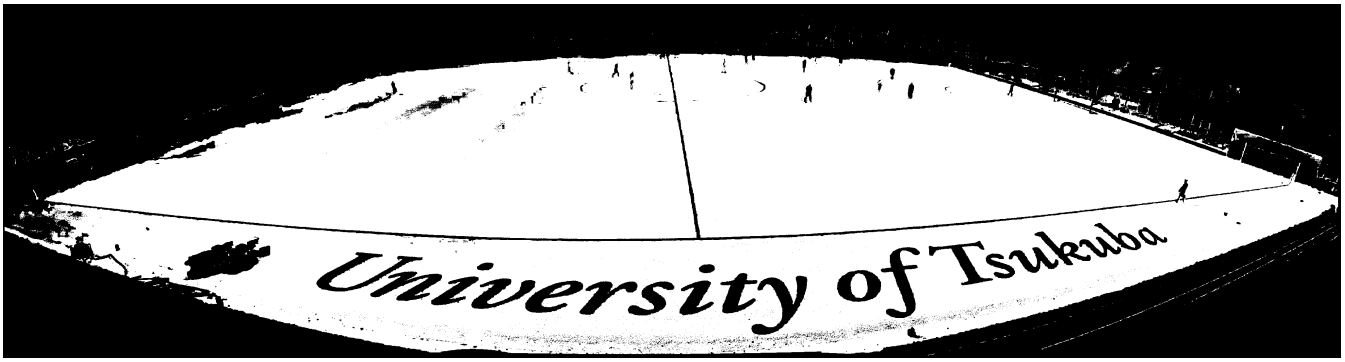
Huit vérifications supplémentaires réussissent : aide, mode invalide ou absent, sortie absente, limites invalides, vidéo absente et essai du mode aucune. Une erreur attendue renvoie un code non nul au lieu de poursuivre avec une option mal interprétée.

Ce que montrent les images

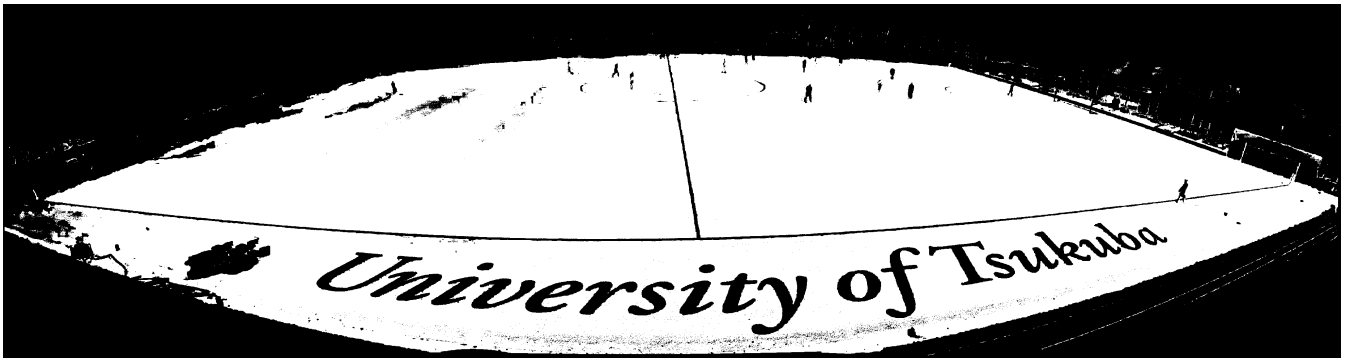
Exemple à l'image 75 du premier clip. Les masques manuel et OpenCV ci-dessous sont les masques binaires complets ; les valeurs 0 et 255 signifient pixel rejeté ou retenu comme pelouse. Les pixels noirs peuvent aussi être des lignes, des ombres ou du décor.



Image originale de la vidéo - clip 1, image 75.



Masque obtenu après normalisation OpenCV - mêmes seuils pour les deux variantes.



Masque obtenu après normalisation manuelle - aucun pixel différent sur cette image.

Pourquoi il faut distinguer validation et amélioration

Sur les neuf captures conservées aux indices 0, 75 et 149 des trois clips, V possède déjà un minimum de 0 et un maximum de 255. La formule min max y conserve donc les valeurs initiales. Cette observation concerne ces neuf captures ; elle n'est pas une mesure de précision de détection.

Les étapes A à D valident l'implémentation et son raccordement. Elles ne démontrent pas une meilleure détection, un suivi plus stable ou une suppression des ombres. L'essai comparatif avec l'absence de normalisation, puis l'évaluation avec le détecteur et le tracker, restent les étapes suivantes.

Reprendre le travail dans VS Code

Lancer la vidéo ou les tests

Ouvre le dossier **S7-MultipistageFootball** situé dans PROJET2A sur le Bureau. Dans Run and Debug, sélectionne **Detourage terrain**, puis F5. Choisis la vidéo et le mode : manuel, opencv ou aucune. Le lancement Debug dispose du même choix. La configuration **Tests mathematiques des traitements** exécute les 32 vérifications sans vidéo.

Depuis un terminal placé à la racine de ce dépôt, après compilation et avec les DLL OpenCV dans PATH, tu peux aussi utiliser :

```
.\build\Release\test_traitements.exe
```

```
.\build\Release\test_traitements.exe `
  --video "C:\Users\amine\Downloads\clip1_fixed_full_pitch_118575.mp4" `
  --frames 150 --output ".\build\resultats_normalisation"
```

Exemple d'exécution du vrai programme, sans fenêtres, avec les exports de vérification :

```
.\build\Release\detourage_terrain.exe `
  "C:\Users\amine\Downloads\clip1_fixed_full_pitch_118575.mp4" `
  --normalisation manuel --sans-affichage --max-images 150 `
  --sortie ".\build\verification_manuelle"
```

Pour ces commandes sous Windows, le dossier des DLL est **PROJET2A\build\vcpkg_installed\x64-windows\bin** ; la configuration F5 le fournit déjà. Utilise un dossier de sortie distinct pour chaque clip et chaque mode afin de conserver les essais.

Les fichiers de preuve

Les journaux, CSV et captures de cette séance sont conservés dans le dossier suivant. Le sous-dossier baseline contient les rendus avant modification, comparaisons les vérifications mathématiques et vidéo, application les exécutions du programme installé.

```
C:\Users\amine\Documents\ChatGPT\Projet2A\tmp\
session-normalisation-2026-10-19\results
```

La suite dans le planning

Le prochain objectif est d'étudier l'utilité du traitement sur des cas de détection identifiés, puis de convenir avec Younes et Badr des entrées et sorties de leurs modules actuels. La chaîne sur un joueur reste le prochain jalon collectif avant le multi-tracking. Le choix d'une accélération FPGA viendra après les mesures de la chaîne complète.

Le travail de cette séance reste **local sur la branche normalisation**. Aucun commit ni push n'a été effectué. Les modifications préexistantes de main.cpp et initiation_opencv.cpp sont conservées ; les nouvelles fonctions et les tests doivent être sélectionnés explicitement lorsque tu décideras de publier ton travail.